DESKTOP-FIRST SAAS FOUNDATION

Frontend Platform: User and Integration Guide

A practical handover for product owners, everyday users, developers and operators.

Reporting period: Monday 7 September 2026, with follow-on work through Tuesday 8 September 2026. All dates and release times use UTC.

Prepared: 8 September 2026. Product: Nexora, a desktop-first frontend foundation for multiple SaaS applications. Current deployed frontend: 20260908020453305-9fbd513a.

This guide describes the inspected working tree and the deployed demonstration. It distinguishes committed Monday history, later workspace changes, and future production integration. It does not claim that every demonstrated business screen is a complete production application. Repository baseline before this documentation: 68d5482.

USER WORKFLOWS • ARCHITECTURE • INTEGRATION • STATUS

Contents

1. What was built, in everyday language

The aim is to build the common frontend once and use it in several SaaS products. A finance application, customer application or other business application should share navigation, tables, forms, saving, import and approval patterns. Its branding, available pages and backend services should belong to that product.

Nexora is the main demonstration. Ledger is a smaller finance example used to prove that a second product can select different branding, pages and role rules. The work prioritizes desktop browsers and desktop windows. Mobile is optional and was not the design driver.

The main improvement is that important controls now perform complete, persisted demonstration workflows. Save acknowledges an actual record save. Refresh asks the service for fresh rows. CSV import validates before writing. Approval records who decided, with comments and a history. These workflows have explicit error, retry and conflict behavior.

What a user can do now

- Open Customer Master, search customers, sort and page through service results.
- Create or edit records, save incomplete recovery drafts, restore acknowledged drafts after reload, and review conflicts instead of silently overwriting another edit.
- Save a personal list layout, rename it, make it the default or delete it.
- Add attachments, comments and links to saved records, and see recent panel activity.
- Upload customer CSV files, map columns, review validation errors, confirm valid rows, inspect progress and retry failed writes.
- Submit saved customers for approval, configure review stages as an administrator, make commented decisions, and track requester notifications.
- Use desktop tabs, keyboard navigation and optional split/detached windows with improved record ownership and logout behavior.

What a product developer gains

Product-owned setup is separated from shared packages. The developer can select existing modules/pages, apply branding and role rules, and replace request, record, panel, import and approval adapters. An adapter is the small piece of code that translates the shared frontend's calls into the product backend's calls.

The completion boundary

Completed here means the shared frontend and its matching persisted demo service implement the documented behavior. Production identity, authorization, databases, object storage, compliance controls, email/push delivery and specialized business rules still belong to the consuming application. Adding an entirely new domain schema or module requires code changes; the starter is not a no-code application builder.

Read sections 4–10 to use the product, sections 11–14 to integrate it, and sections 15–19 to run, verify and plan it.

2. What happened on Monday, and what followed

“Last Monday” means 7 September 2026. Some work continued after midnight into Tuesday. The following distinction prevents Tuesday's additions from being presented as Monday commits.

Monday: confirmed in Git history

Evidence	Change	Why it matters
3d5efa8, 01:07 UTC	Shared filter bar adopted by reports; reference-data failures wired into controls; inline-edit conflicts connected	Users see a failed lookup or conflicting edit instead of an empty list or a misleading success
c99ac8e, 02:20 UTC	Filled-color contrast corrections and browser/API preflight checks	Text on colored controls is more readable, and browser tests verify the API they actually target
c4e6a54, 02:44 UTC	Deployment build guard	A developer's local build cannot silently replace the files served publicly
63bceff, 12:31 UTC	Ranked remaining-work document	Incomplete work and external dependencies became explicit
e055a24, 15:55 UTC	Shared checkbox, radio and file-picker controls; label/hint correction	Forms use common behavior and accessible labels
b07163c and merge 68d5482	Structural guard for shared form controls	Future changes are checked for drifting back to separate control implementations

These commits are repository history. Their author metadata is retained in Git; the table does not attribute all historical changes to a single assistant session.

Monday: subsequent development and deployment evidence

The working tree and dated release notes record further desktop-first work on Monday. It includes clearer headings and new-record state; focus restoration; document-owned dirty state and AI sources; session/logout and detached-window cleanup; safe preference initialization; durable record/draft editing; real worklist paging/refresh/archive; personal views; and conditional/dependent form validation.

Release	Main milestone
20260907124618130-3b6cb502	Isolated frontend release activation

Release	Main milestone
20260907152207170-63028a2b	Desktop layout, initial product profile and reliability work
20260907163215381-b9dabe79	Coordinated record-saving/recovery service and frontend
20260907172055053-faace50b	Worklists, personal views and form-rule workflows

These later changes were present in the working tree at the start of this documentation task; they should not be mistaken for additional Monday commits already present on origin/main.

Tuesday: follow-on additions

Release	Addition
20260908002211440-aa85e69e	Product starter, application-owned composition and Ledger checks
20260908004731762-bb1d1b06	Reusable attachments, comments, related records and panel activity
20260908011625901-fddb7b51	Customer CSV upload, mapping, validation, confirmation and resumable results
20260908020453305-9fbd513a	Configurable customer approvals, bulk decisions, history and in-app requester notices

The latest release above was active on both public shells when this guide was prepared. Deployment is an artifact snapshot; it is separate from Git commit/push status. Older documents contain historical “active release” statements and test totals. Treat those as dated evidence, not current status.

3. Completion map

Use this table to distinguish a usable frontend workflow from a finished production integration.

Area	Completed in this foundation	Still outside completion
Shared shell and themes	Header/sidebar, module navigation, command palette, common controls, fourteen themes and desktop personalization	Full localization/RTL acceptance and every screen/theme combination
Desktop workspace	Tabs, targeted keyboard fixes, split examples, document-owned edits and sources; Linux native lifecycle checks	Windows/macOS native validation, physical multi-monitor coverage

Area	Completed in this foundation	Still outside completion
Product starter	Branding, existing page/module selection, role-filtered navigation/actions, Ledger example, generator	Arbitrary module/schema registration, complete product packaging and independent browser storage namespaces
Record editing	Versioned save/create, service-backed drafts, restore/discard, retry, conflict review; forms/billing/consultation adoption	Offline crash protection, business posting/signing, automatic merging
Worklists	Server paging/filter/sort/count, actual refresh, stale-response protection, partial archive results	Production record/inline-edit reconciliation and select-all-matching bulk operations
Personal views	Create, apply, rename, delete and default; account-owned persistence	One unified personal/shared view management experience
Form rules	Required/conditional fields, dependent options, ranges, date comparisons and server field errors	General repeatable sections, async lookup/custom renderer registry, complete localization
Record panels	File upload/download/removal, comments, related links, recent panel activity	Large/resumable transfers, malware scanning, durable unsent comments, complete record audit
Customer CSV import	All five requested steps plus error CSV, safe batches and failed-row retry	Other entity definitions, large-job workers and distributed database uniqueness
Customer approvals	All five requested steps plus stale-version protection and idempotent decisions	Other entities, workflow side effects, global bell synchronization, email/push
Deployment	Isolated builds, artifact checks, activation and frontend rollback	Zero downtime, native installers, infrastructure provisioning and automatic API migration
AI controls	Existing gates/classification and improved per-document source ownership	The ten server hardening items in section 17

Existing billing, consultation, spreadsheet, reporting, inbox and library screens remain valuable examples. Their presence does not imply payment processing, invoice posting, clinical signing, a complete reporting backend or a unified production notification service.

4. Start here: a new user's first session

The web site is https://front-design.pepbits.com. The desktop browser shell is https://desktop.front-design.pepbits.com. A browser desktop shell and an installed native Tauri application share much of the interface, but they are different delivery modes.

1. Sign in with an account supplied by the application administrator. Do not put production passwords or service credentials in this guide or product source.
2. Use the sidebar to open Customer Master. Alternatively, press Ctrl+K, type Customer Master and choose the result.
3. Search the list or open a record using its Edit action. A record is one saved business item, such as a customer.
4. Make a small permitted change and save it. Wait for the saved acknowledgment. “Not saved” or a pending recovery indicator is not a completed save.
5. Return to the list and use Refresh if you need the latest server result.
6. Open Approval inbox to see requests available to your role. Use Import records in the worklist's More actions menu when creating customers from CSV.

Who does what in the demonstration

Role	Typical use
enterprise-admin	Configure approval stages, administer available product features and act as an approver when the stage includes this role
finance-manager	Write record panels/import customers and review stages assigned to finance-manager
operations-analyst	Read permitted screens and panels; submit customers for review; decide only stages explicitly configured for this role, never their own request

The demonstration includes admin, user1 and user2 account names corresponding to these roles. Obtain credentials from the administrator. Product profile rules may make page/action availability stricter. Backend permission checks remain authoritative.

When an action is unavailable

A disabled action may need a saved record, selected rows, a comment, a successful data load, or a permitted role. In approvals, a requester cannot approve their own submission. A role may see a request history without being allowed to decide its current stage.

5. Desktop workspace and preferences

The design assumes a desktop screen, keyboard and pointer. Persistent navigation and dense lists are intentional. The work checked common desktop widths including 1280, 1440 and 1920 pixels, plus selected split-pane behavior. A narrow pane may scroll a table or reduce secondary information.

Working with more than one record

1. Open the first record, then open another record in the desktop workspace.
2. Each record owns its edits and recovery state. Saving a background record must not mark the active record clean.
3. Use document tabs to switch. Arrow, Home and End support tab navigation; Delete can close a tab. The active document also has an accessible close control.
4. Alt+S belongs to the active form. Repeated save shortcuts should not create duplicate writes.
5. If configured, use split panes or detach a record into a native window. These are optional desktop capabilities, not required mobile behavior.

AI sources were also scoped to the owning document and record. Switching documents should not cause a helper to read another record's selected sources.

Closing, logging out and recovery

Closing with discard waits for an in-flight recovery operation before removing its draft. If removal fails, the interface explains that recovery remains. Logout removes the workspace and invalidates detached sessions. Linux native lifecycle checks covered detach, close/reattach and logout propagation; other operating systems still require their own checks.

Preferences are written only after a successful initial read and an explicit edit. A failed initial load shows a notice and prevents accidental default settings from overwriting stored settings. Preference-save retry/conflict experience is still separate follow-up work.

Focus and readability

Page headings take priority over secondary header text. Opening and closing a command palette or modal should return keyboard focus to the trigger. Nested dialogs give keyboard handling to the top dialog. These changes have targeted coverage; a complete keyboard and screen-reader audit of every screen remains pending.

6. Create, save and recover a record

A business code, such as a customer code, is not the record's internal identity. New form records receive a separate identifier. This allows codes to be edited without changing the record identity used by drafts, links and panels.

1 ↓	Open saved record or New
2 ↓	Load saved values and any acknowledged recovery draft
3 ↓	Restore draft, discard draft, or start editing
4 ↓	Validate and Save
5	Acknowledge saved version, or review error/conflict and retry

Normal save

1. Open an existing record or select New.
2. If recovery is offered, choose Restore draft or Discard recovery draft deliberately. Restoring a draft does not save the record.
3. Complete the required visible fields. Changing a parent choice can clear an invalid dependent selection. For example, changing country can invalidate the selected state.
4. Select Save. The service checks the expected version and applicable form rules.
5. On success, the interface records the acknowledged save time and version. Edits typed while that request was running remain unsaved.

Recovery draft versus saved record

After roughly 800 milliseconds without another edit, the editor queues a recovery write. Save draft requests it immediately and permits incomplete forms. Acknowledged service drafts can survive reload/restart. Text still in the debounce interval, in a failed request or typed offline is not guaranteed to survive a crash.

Record contents are not stored in browser localStorage, preferences or workspace restoration metadata. Recovery is provided by the service adapter. Drafts belong to an account, tenant, product and record; saved records are shared within the relevant tenant/product scope.

Failure and conflict handling

- For a connection failure, keep the editor open and retry. The same operation identity is reused where the result is uncertain.
- For a field validation failure, correct the highlighted field and save again. Server field errors are attached to controls and the relevant section is opened.
- For a version conflict, review the latest saved values and any competing recovery draft. Choose the saved version or explicitly retain your edits for a later save. There is no automatic merge or silent overwrite.

Billing and consultation use the same controller. Save invoice is not invoice posting or payment processing. Save consultation is not clinical signing; incomplete documentation can still be saved. Those business actions need their own service rules.

7. Lists, personal views, archive and export

A worklist is the table or card list used to find and act on records. Search, filters, sorting, page number and page size go to the service. It returns one page and the matching total count.

1 ↓	Choose search, filters and sort
2 ↓	Request one page from the service
3 ↓	Show rows and matching count
4 ↓	Select rows on this page and perform an action
5	Show each result, then refresh acknowledged changes

Find and refresh records

1. Enter a search value or change a filter.
2. Choose sort order and page size if needed.
3. Move through pages using the list controls. Old requests are cancelled or ignored when a newer request supersedes them.
4. Select Refresh to read current service data. A failed request shows an error with retry; generated sample rows are not presented as a successful live response.

Selection means rows on the current page. Changing query, order or page clears selection. “All matching records across every page” is not implemented as the general bulk selection contract.

Save a personal view

1. Set filters, visible columns, sort and page size.
2. Open Saved views and use Save current view.
3. Name the view. Later, apply it, rename it or mark it as default.
4. Reopen the list to use its default, unless an explicit shared view or intervening user interaction takes precedence.
5. Delete a view when no longer needed. Deleting a view does not delete its records.

Views belong to an account, tenant, product and page. The demonstration allows 50 views in a scope and names up to 80 characters. Concurrent stale changes return a conflict and require reload. Existing opaque shared-filter links are separate from these personal layouts.

Archive and export

Archive returns a result for every selected ID. Successful rows disappear after refresh; failed rows remain available for review/retry. Unknown or locked rows can fail individually. Archive is not permanent deletion or a production retention workflow.

Export and print cover the current page or explicit selection as described by the interface. Classification rules refuse credential/unclassified columns and require review for sensitive content. A durable production export audit is still pending. Inline cell editing remains a demonstration session store; a production integration must reconcile it with versioned full-record saves.

8. Attachments, comments, related records and activity

These panels belong to a saved record. They save separately from the form. Saving or discarding the form does not post an unfinished comment, remove a file or create a related link.

Attach a file

1. Open a saved record and select Attachments in Record supporting information.
2. Choose a file between 1 byte and 2 MB. Wait for the successful attachment listing.
3. Use Download to retrieve the authenticated file. The demo downloads bytes rather than rendering uploaded HTML or SVG inside the application.
4. Use Remove and confirm to remove an attachment.

There are at most 20 attachments per record in the demo. File contents are stored as base64 in a private JSON file. Production applications should replace this with authorized object storage, scanning, transfer progress/resume and retention rules.

Add a comment or relationship

Comments are plain text, attributed to a user and time, with a 4,000-character limit. Post comment saves it. The author or an administrator may remove it. The demo allows 200 comments per record.

For a related record, choose a configured page, enter an existing record ID and a relationship label, then Link record. Open follows the shared navigation interface. Self-links and duplicate links are rejected. Up to 100 links are supported per record in the demo.

Understand activity and retries

Activity shows successful panel changes, newest first, with up to 500 entries. It is not the complete record-save or approval audit. Approval decisions have their own history.

If a mutation response is lost, Retry change preserves the operation identity and avoids duplicating an acknowledged change. A correctable rejection keeps inputs available for correction. Comment/link inputs survive desktop tab suspension in memory, but unposted text is not durable across document close or reload.

Panel writes require both product edit availability and service permission. The demo allows finance-manager and enterprise-admin to write; other authenticated roles can read permitted records.

9. Import customers from CSV

Open Customer Master, then More actions → Import records. This workflow creates new customers; it does not overwrite existing customers.

1 ↓	Upload CSV and preview the first five rows
2 ↓	Map columns to application fields
3 ↓	Validate every row, including duplicates
4 ↓	Review errors and explicitly confirm valid rows
5	Import in batches, inspect results, resume or retry failed writes

Prepare the file

Use a UTF-8, comma-separated .csv file with a header row. Limits are 2 MB, 500 data rows and 80 unique, non-empty headers. Quoted commas, quoted multiline cells, escaped quotes, a UTF-8 BOM and Windows line endings are supported. Blank rows are ignored. Error row numbers count data rows, excluding the header and blank rows.

Example columns and one example row:

```
customerCode,legalName,customerType,contactName,email,address1,city
DEMO-101,Example Co,Corporate,Contact,hello@example.com,Street,Dubai
```

Use fictional data while testing. Customer fields include customer code, legal name, customer type, contact name, email, address, city and country. Unmapped fields use their schema defaults, including country AE in the demonstration. A mapped empty cell stays empty and can fail required-field validation.

Map and validate

Matching field IDs or labels are suggested automatically. Select the correct CSV column for every required field without a default. One source column can map to at most one application field. Select Validate mapped rows to check all rows on the server.

Validation covers required/conditional fields, email, supported option values, numeric rules, dependent choices and applicable dates. Numbers exclude grouping separators and currency symbols. Toggles accept true/false, yes/no or 1/0. Multiselect values use semicolons. Dates use YYYY-MM-DD where applicable.

Customer codes are compared after trimming and case folding. Every occurrence of a duplicate inside the file is invalid. Codes already in saved or seeded customers are invalid too.

Confirm and inspect results

1. Review each row's errors, switch to Error rows, or download the error report.
2. Change the mapping if needed. Correct bad source values in the original CSV before validating a new import.
3. Choose Import valid rows, then explicitly confirm. Validation alone does not create records.
4. The service processes ten pending rows per batch. Progress distinguishes ready, imported, invalid and failed rows.
5. Closing pauses after the current batch. Reopening or reloading restores the latest server job. Resume continues pending rows. Retry failed rows retries transient failures without submitting successful rows again.

A code created after validation becomes a non-retryable row error. Correct the CSV and validate again. Deterministic job/row record IDs recover safely when a record was saved but its receipt or response was lost.

Only the latest job is exposed in this UI. Finish pending work and retry transient failures before starting another job. The demo retains at most 20 jobs per user/product/page, preserving unfinished confirmed jobs. The error report identifies rows and field errors; it is not a complete corrected copy of the source CSV. Other entities and larger background imports remain integration work.

10. Submit, review and approve customers

A request has a requester, a saved record version and one or more ordered review stages. A stage contains the roles allowed to decide it. A requester can never approve their own request, even if they are an administrator.

1 ↓	Requester saves the customer and submits with a comment
2 ↓	Reviewer inspects the saved record at the current stage
3 ↓	Approve advances a stage; reject or request changes ends this review
4 ↓	Requester can correct, save and resubmit for a new review
5	Final approval or rejection appears in history and requester notices

Administrator: configure the stages

1. Open Customer Master → Approval inbox → Configure stages.
2. Enter a stage name and choose one or more approver roles.
3. Add, remove or order stages by their displayed sequence; the UI appends new stages and removes selected stages. To change order, edit the stage entries accordingly.
4. Use one to five stages, then Save approval stages and confirm.

The default is Finance review, assigned to finance-manager or enterprise-admin. New configuration applies to future submissions and resubmissions. Existing pending requests retain the stages captured when they were submitted. Configure an available approver other than the requester. Account-specific assignment, delegation and escalation are not implemented.

Requester: submit or resubmit

1. Open the saved customer. Save any form edits first.
2. In Record approval, add an Approval comment.
3. Select Submit for approval and confirm.
4. After requested changes or rejection, edit and save the record as needed, add a new comment, then Resubmit saved record.

The original requester owns resubmission. Submitting an unchanged version already pending or approved is refused. Changing the saved record during review prevents decisions until

resubmission. A visible warning identifies an earlier approval when the current saved record has changed. Approval does not lock the form or execute downstream business actions.

Reviewer: individual or bulk decisions

Open a record to review its details. If your role can decide its current stage, enter a comment and choose Approve, Reject or Request changes, then confirm. Approve advances one stage; approval at the last stage marks the request approved.

For bulk decisions, open Approval inbox. Filter by status, current stage, ownership or record/requester search. Select eligible rows on the displayed page, write the bulk comment, choose the decision and confirm. The page shows 25 requests at a time; the service accepts at most 100 distinct records per action. Every result is checked independently, so a stale or unauthorized row can fail without undoing other successful decisions.

Refresh approvals before acting on information changed by another user. A version conflict requires reviewing fresh data. An uncertain response exposes Retry approval action, which reuses the same operation identity.

History and requester notifications

Expand Approval history for a record to see decisions, comments, actors, stages and timestamps. Earlier review cycles remain after resubmission.

Expand My approval notifications in the inbox or record section. Requesters receive persistent in-app notices for submission, stage advancement, final approval, rejection and requested changes. Open a notification to inspect the record. Mark notifications read updates the stored read state. The latest 100 notices are displayed and changes from other users appear on refresh.

These notices are specific to approvals. Email, push, automatic live delivery and synchronization with the general header notification bell are pending integrations.

11. Technical architecture, explained simply

The repository is a monorepo: several applications and shared packages live together and are built/tested together. The two shells present the same shared screens through different navigation and window implementations.

1 ↓	Product setup: products/active.ts selects branding, pages and services
2 ↓	Web shell: Next.js | Desktop shell: Vite and optional Tauri
3 ↓	Shared shell and screens: navigation, workspace, forms and workflows
4 ↓	Shared data controllers and product adapters
5	Demo API today | Application backend through replacement adapters

Layers and responsibilities

Layer	Main location	Responsibility
Product composition	desktop-clients/products	Select a product and inject its services inside the session provider
Browser shells	apps/web and apps/desktop	Next routing or desktop SPA/window behavior; build-time API URL
Navigation/window abstraction	packages/platform-ports	Screens ask to open a target without depending on browser or native routing
Desktop state	packages/workspace-core	Document identity, lifecycle, tabs, owned state and draft coordination
Shared chrome	packages/erp-shell	Product context, header/sidebar, command palette and workspace presentation
Shared screens	packages/erp-screens	Forms, worklists, record panels, imports, approvals and domain examples
Configuration	packages/erp-config	Existing navigation registry, product definitions, schemas, form rules and classification
Data contracts	packages/erp-data	Record controller, adapters, CSV parsing/mapping and workflow interfaces
UI and themes	packages/ops-ui and packages/tokens	Common controls, overlays and semantic colors/spacing
Session and AI	packages/auth; ai-config, ai-client, ai-ui	Session behavior and separate AI configuration, request and presentation layers

Layer	Main location	Responsibility
Demo service	dummy-api/server.mjs and store modules	Authenticated demonstration routes and persisted JSON workflows

The inspected stack uses React 19, Next.js 16.3.3, Vite 7, TypeScript and Tauri 2. Node 24 is used by the build/deployment and the demo server. Versions are a repository snapshot, not an instruction to upgrade to whatever is newest.

A request through the layers

When a user confirms an approval, the shared screen calls ApprovalAdapter.change. The default adapter calls the product request transport. Authentication attaches the session, the demo API checks roles/version/record state, and the approval store atomically writes the result. The response updates the screen. A replacement adapter can call another backend while keeping the same user experience.

The browser is not the authorization boundary. Hiding an action improves usability; the backend must independently authorize the request. Tenant identity comes from the authenticated session, not a tenant value supplied by a client form.

12. Data ownership and service contracts

How records are separated

A tenant is a customer organization. A product is an application profile. A page identifies a registered record type or screen. A record ID identifies a saved item. An account identifies the signed-in person.

Data	Scope and lifetime
Saved records	Shared within the appropriate tenant/product; versioned snapshots
Recovery drafts	Account + tenant + product + record; only acknowledged drafts survive a crash
Personal views	Account + tenant + product + page
Panels	Tenant + product + page + saved record; comments have author ownership
Import jobs	Tenant + account + product + page; latest-job UI and bounded demo retention
Approvals	Tenant + product + page; requester/stage-role visibility; notifications restricted to recipient

Data	Scope and lifetime
Browser workspace metadata	Navigation/layout/identity; not durable storage for record contents

The record key encodes [productId, editorType:pageId:recordId]. An example form key is ["acme-finance", "form:customer-master:record-123"]. The serialized key is URL-encoded when used in a record route. Import IDs and approval operation IDs support safe retry; they are not user-facing business codes.

Main HTTP contracts

Paths below are relative to the configured API base. Authentication is required.

Endpoint	Purpose and important behavior
GET /records/:key	Return saved record, recovery draft and draft version
GET /records?scope=...	List saved record snapshots in the requested scope
PUT /records/:key	Save values with expected record/draft versions and operationId
PUT /records/:key/create	Create at destinationKey; clear the source new-record draft atomically
PUT /records/:key/draft	Save recovery values with baseVersion and expected draft version
PUT /records/:key/discard	Discard at the expected draft version; return 204
POST /worklists/search	Search/filter/sort/page with a matching total
POST /worklists/archive	Return one success/failure result per selected ID
POST /personal-views	List, create, rename, delete or set default with version checks
POST /record-panels	List/change/download using a product/page/record scope
POST /imports	preview, latest or run; run may retry failed rows
POST /approvals	read, configure, submit, decide or mark-read

A record snapshot contains values, a positive version and savedAt. A recovery snapshot also carries baseVersion. Return 409 for an expected-version conflict and appropriate field errors for rejected values. The form service uses 422 for schema validation failures. Preserve the documented response shape and return machine-readable errors.

Safe retries and storage

An operation ID identifies one intended mutation. Retrying after a lost response uses the same ID and the same payload. Reusing an ID for different data is rejected. This is called idempotency: repeating the same request does not repeat the business effect.

The demo stores records.json, workspaces.json, record-panels.json, imports.json and approvals.json under the record data directory. NEXORA_DATA_DIR moves the general demo data directory; RECORD_DATA_DIR overrides record-related storage. JSON writes use a temporary file and rename. This is a single-process demonstration, not a multi-server transactional database.

Records retain the latest 200 idempotency results per record; panels retain 500. Approval history/receipts are retained without an automatic pruning policy. Production adapters must define storage, backups, retention, encryption, concurrency and migrations explicitly.

13. Use the foundation for a different application

There are two different integration jobs. Reusing existing pages with different branding and data is supported by the starter. Introducing a completely new domain schema, module or workflow requires extending the registries and backend adapters.

Path A: select existing capabilities

From desktop-clients, with Node 24 and dependencies installed:

```
npm run product:create -- --id acme-finance --name "ACME Finance" --module finance
```

The generator creates product.ts, services.ts and setup instructions under products/acme-finance. It refuses an existing folder and does not change the active product. Generated starters restrict writes/exports to enterprise-admin until roles are configured.

Edit product.ts using the existing registry:

```
import { defineProduct } from "@pepbits/erp-config";

export const product = defineProduct({
  id: "acme-finance",
  name: "ACME",
  accentName: "FINANCE",
  tagline: "Finance workspace",
  defaultModule: "finance",
  enabledModules: ["finance"],
  enabledPages: ["customer-master"],
  pageTitles: { "customer-master": "Customers" },
  access: {
    actions: {
      create: ["enterprise-admin", "finance-manager"],
      edit: ["enterprise-admin", "finance-manager"],
      archive: ["enterprise-admin"],
      export: ["enterprise-admin", "finance-manager"],
    },
  },
});
```

Select it in products/active.ts for your application checkout:

```
export { product } from "./acme-finance/product";
export { services } from "./acme-finance/services";
```

Both shells consume that choice through products/provider.tsx inside SessionProvider. Keep Nexora selected in the shared public demonstration unless intentionally releasing another product. enabledPages limits domain pages while retaining shared utilities and relevant dashboards. Unknown registry IDs are rejected. Omitted access rules retain existing demo availability; an empty role array denies an action to everyone.

Path B: add a new domain

For a service-desk ticket application, for example, branding alone does not create Ticket Master. A developer must add or extend module/page types and navigation registration, define the ticket schema/reference options, provide worklist data mappings, and implement authorized ticket persistence. Customer-specific import uniqueness/save mappings and approval record resolvers must be replaced or extended for tickets. Register the UI entry points only after the service supports them.

Use the existing adapters and screens where their contracts fit. Keep specialized behavior in product/domain code. Generic schema registration, custom field renderers and fully arbitrary modules are still planned extension points, so this step currently requires shared registry code changes and regression checks.

Deployment and upgrade ownership

Use separate application origins while the demo authentication/preferences storage keys remain shared. Give each product its own API configuration, native bundle identity, icons, metadata, legal text and document branding. The profile does not automatically replace every invoice/clinical fixture string.

Keep application setup in products/<id> and shared code in packages. Prefer shared-package upgrades in the monorepo over copying each component into unrelated repositories. If copying the repository, record the upstream commit, keep product changes isolated and review adapter/schema migrations on every update. Independently versioned package publishing and complete migration tooling remain pending.

14. Connect a real backend

The public ProductServices interface currently exposes request, records, panels, imports and approvals. They are optional because the demonstration supplies defaults. Authentication, preferences, general notifications and AI still use their existing separate boundaries.

Option 1: compatible HTTP API

If the product backend implements the documented paths and shapes, keep the default adapters and point the shells to that API through deployment configuration. A backend-for-frontend can translate those routes to internal services while the browser keeps a compatible contract.

Option 2: product-owned adapters

Use services.ts to supply domain adapters. The following is a composition pattern: the named adapter modules are application code that you implement, not files already provided by the starter.

```
import type { ProductServices } from "@pepbits/erp-screens";
import { records } from "./adapters/records";
import { panels } from "./adapters/panels";
import { imports } from "./adapters/imports";
import { approvals } from "./adapters/approvals";

export const services: ProductServices = {
  records,
  panels,
  imports,
  approvals,
};
```

A compatible transport can also reuse the current session helper:

```
import { authedFetch } from "@pepbits/auth";
import type { ProductServices } from "@pepbits/erp-screens";

export const services: ProductServices = {
  request: (path, init) => authedFetch(path, init),
};
```

This last example keeps the demo session protocol. Replacing identity with another authentication system requires adapting the auth/session boundary too; a new API URL alone does not perform that integration. Preserve AbortSignal when forwarding requests and do not convert failed responses into successful empty arrays.

What each adapter must implement

Contract	Required methods	Preserve
RecordAdapter	list, load, create, save, draft, discard	Version conflicts, compatible value shapes, ISO times, durable acknowledged drafts, idempotency
RecordPanelsAdapter	load, change, download	Authorized identity lookup, file bytes/metadata, ownership, limits and retry identity
ImportAdapter	preview, latest, run	No writes during preview; per-row validation/status; duplicate rules; safe batch recovery

Contract	Required methods	Preserve
ApprovalAdapter	read, change	Stage roles, requester exclusion, saved-version checks, history, notifications, partial bulk results
request transport	(path, init) returning Promise<Response>	Session ownership, status/body contracts, cancellation and product-specific routing

ImportAdapter.preview receives productId, pageId, mapped rows and a job ID. latest returns a job or null. run receives the job ID and an optional retry flag. ApprovalAdapter.read receives [productId, pageId] and an optional record ID; change additionally receives a typed action and operation ID.

Integration acceptance flow

1 ↓	Define product pages, roles and domain field mappings
2 ↓	Implement authenticated service adapters and schema migrations
3 ↓	Verify success, rejection, conflict, stale response and lost-response retry
4 ↓	Exercise two users and two tenants with isolated fixtures
5	Build both shells, prepare a release and run product smoke checks

Enforce uniqueness in a transactional database for imports. Store files in an authorized object service. Apply record-level permissions on every endpoint, including download and history reads. Derive tenant identity from the server session. Do not place provider/database secrets in frontend source or public environment variables.

For approvals, decide whether approval locks a record, permits later edits, triggers downstream actions or requires account-specific assignments. The current demo approves a saved version and warns when it changes; it does not post an invoice, authorize a payment or sign clinical documentation.

15. Run locally and verify changes

Use Node 24. The documented commands assume a checkout containing the developed source, not just a historical baseline. The documentation task does not itself make uncommitted application files appear in a different checkout.

Install and start an isolated test stack

From the repository root:

```
cd desktop-clients
npm ci
```

Start the API in one terminal, from the repository root, using a new test directory:

```
NEXORA_DATA_DIR=/tmp/acme-demo-test \
RECORD_DATA_DIR=/tmp/acme-demo-test \
PORT=3330 node dummy-api/server.mjs
```

In another terminal, from desktop-clients:

```
VITE_API_URL=http://localhost:3330 \
npm run dev -w desktop -- --host 127.0.0.1 --port 3109 --strictPort
```

For the web shell, use a separate port and set its public API URL when starting development:

```
NEXT_PUBLIC_API_URL=http://localhost:3330 \
npm run dev -w web -- --port 3110
```

Check the browser's requests to verify that it calls the isolated API. Public API URLs are compiled into production assets; changing only a server-start environment variable does not rewrite an existing production bundle.

Verification commands

From desktop-clients:

```
npm run typecheck
npm test
npm run test:deployment
npm run test:products
npm run test:records-api
npm run test:workflows-api
npm run test:panels-api
npm run test:imports-api
npm run test:approvals-api
npm run build
npm run verify
```

Several API scripts intentionally overlap the shared HTTP integration test. A faster complete API run from the repository root is node --test dummy-api/*test.mjs. Build before artifact-dependent verification when new utility classes or API URLs have changed; stale local CSS is not a valid check of new source.

The feature browser scripts include e2e:records, e2e:workflows, e2e:panels, e2e:imports and e2e:approvals. They create, modify or archive fixtures. Run them only against the isolated frontend/API pair. Set E2E_DESKTOP to the frontend and E2E_API where the suite uses it. PLAYWRIGHT_PATH can point to an existing Playwright installation. The runtime also needs Chromium's host libraries.

Ledger's product-starter browser test requires temporarily selecting Ledger in a local test checkout. Restore the intended product before building a release. Native Tauri checks require a Rust/native toolchain and their own platform setup; a browser check does not replace a native installer/device test.

16. Release, operate and roll back

The production frontend serves a selected artifact under .deploy/releases. Ordinary workspace builds do not change that selected artifact. The currently reported sites use frontend ports 3100 and 3101 behind a reverse proxy, with the API on 3200.

Prepare and activate

From desktop-clients:

```
cp deploy.config.example.json deploy.config.json
```

Set webApiUrl and desktopApiUrl to public browser-reachable API addresses. The example domain must be replaced. Keep secrets out of this file. The config is intentionally environment-owned and ignored by Git.

```
npm run deploy:prepare
npm run deploy -- --activate RELEASE_ID
```

Use the actual ID printed by preparation. Preparation snapshots the frontend, installs the lockfile, builds both shells with explicit API URLs, verifies assets, packages standalone runtimes and tests them on isolated ports after removing the temporary build. It does not change running services.

Activation changes the selected release, restarts web and desktop, then checks release identity, HTML and assets. It is a short service restart, not zero-downtime deployment. A failed activation attempts restoration of the previous frontend. No previous release exists on a first deployment.

Coordinate API changes separately

The frontend deploy command does not deploy or restart the API. For batches that add endpoints, the operator must prepare a compatible API, preserve private data/source backups, preserve the service environment, restart the API and verify it before activating a frontend that depends on it. Restarting this demo API invalidates in-memory sessions, so users sign in again.

Do not expose backup contents in logs or commit them. An API rollback may require compatible source and data/migration handling; switching a frontend symlink alone is not an API rollback.

Check and roll back

Check /__nexora-release.json on both frontend hosts, load their HTML/assets, check API health, then make authenticated read-only checks of the changed workflow. Keep fixture-writing browser suites off the live shared deployment.

```
npm run deploy -- --activate PREVIOUS_RELEASE_ID
```

At this handover, the current frontend is 20260908020453305-9fbd513a and the immediately preceding frontend is 20260908011625901-fddb7b51. Retained artifacts and private API/data backups support operator rollback. No automatic pruning policy is assumed.

For parallel deployment tests, use different frontend ports and a different NEXORA_RUN_DIR so test PID files cannot control production processes. Keep NEXORA_DEPLOY_ROOT outside desktop-clients. Native installers, reverse-proxy provisioning, certificate renewal and backend infrastructure are separate operational tasks.

17. Pending work, owners and acceptance criteria

These are proposed priorities based on the inspected foundation. They are not delivery estimates or claims that production integration is already complete.

Product and frontend backlog

Priority / owner	Pending work	Done when
P1 / platform frontend	Arbitrary module/schema registration and custom field renderers	A new domain can register its own schema/module without editing a central domain switch
P1 / app + platform teams	Remaining auth, preferences, general notification and file boundaries	Products can replace these services consistently and isolate browser state by product/account
P1 / frontend	Unified header/inbox notifications	Reading a notice updates the same unread state everywhere, with loading/error/retry behavior
P1 / app teams	More import/approval entities	Each new entity has a schema, identity resolver, uniqueness rule, authorization and tested write flow
P1 / backend + frontend	Reconcile inline edits with record saves	List and form edits share one authoritative concurrency model
P2 / frontend	Richer form fields and async lookups	Repeatable sections, date/time/currency rules and async options have reusable contracts

Priority / owner	Pending work	Done when
P2 / frontend + design	Complete desktop keyboard/theme/RTL coverage	The agreed screen matrix passes keyboard, focus, screen-reader and layout checks
P2 / desktop team	Windows/macOS and multi-monitor validation	Native detach/reattach, logout, close and display changes pass on supported devices
P2 / frontend	Preference-save retry/conflict UX	Failed or conflicting preference saves can be resolved explicitly
P2 / frontend	Unified personal/shared views	Users manage shared filters and personal layouts through a coherent workflow
P2 / app + platform teams	Durable unsent panel input and large file transfers	Unsent input recovers after restart; transfers can report progress and resume safely
P2 / platform team	Performance and distribution	Startup/bundle measurements, optional-module loading, package versions and migrations are documented and tested
Optional / product owner	Mobile and broader component catalogue	Product requirements specify acceptance coverage and demonstrable component states

The old roadmap contains competing opinions about a separate component sandbox. The newer multi-product review suggests extending the existing Library first. Treat the direction as a product decision, not a completed feature. Minor navigation questions about the shared module branch and dashboard special case also remain in the older backlog.

Production workflow work

Replace single-process JSON storage with a transactional service and define retention, backups, migrations and per-record permissions. Add import job workers and transactional duplicate constraints for large/multi-server imports. Add file scanning, authorized object storage and download auditing. Define durable audit history across full record edits, exports and workflow events.

Approvals need product decisions for delegation, escalation, account assignment, deadlines, locking and downstream side effects. Requester notices currently arrive through in-app reads/refresh; email/push and the global bell are not connected. Billing payments/audit previews and clinical signing are not implemented business services.

AI hardening: all ten ledger items remain open

ID	Current gap	Required production owner/action
N1	Provider credentials in a private plaintext file, no vault	Platform/security: managed secret storage
N2	No encryption-at-rest layer	Platform/security: encryption and external key management
N3	Rotation timestamp without enforced rotation policy	Operations/security: expiry, rotation and overlap rules
N4	Operational console lines instead of a durable audit store	Backend: queryable retained audit events
N5	No server-side identifier stripping before provider egress	Backend/security: validate/redact by data class before provider calls
N6	No approved-provider endpoint allowlist	Platform/security: deployment/data-class allowlists
N7	No established provider agreement or region guarantee for sensitive clinical use	Product/legal/platform: required provider and regional controls
N8	Open CORS in the demonstration	Operations/backend: deployment-specific origin allowlist
N9	Demonstration role checks rather than the platform authorization model	Identity/backend: production permissions and policy enforcement
N10	Tenant-level limits without per-user budgets	Backend: user limits within tenant limits

These are repository ledger findings, not legal or compliance advice. The client gates and classification controls do not close server-side gaps. No provider secrets or real sensitive business records are included in this document.

Credential-backed speech adapter checks remain unverified without the required accounts. The older operations notes also list credential rotation and certificate-renewal integration; their completion was not verified for this documentation task. Assign an operator to confirm them before relying on them.

18. Validation evidence and troubleshooting

What the latest development checks establish

Evidence	Result and scope
Full frontend suite in approval batch	1,339 tests passed across 76 files before the final additional changed-record warning test
Final targeted approval tests	Three tests passed, including the added warning test; no new full-suite count is invented
API suite	All 20 store/HTTP tests passed after the final server change
Typechecks and builds	Package typechecks, Next/Vite production builds, structural verification and packaged HTML/asset checks passed
CSV browser journey	Fifteen rows: twelve valid rows imported in two batches, three excluded; report download, restored results and repeat-import duplicates checked
Approval browser journey	Admin configuration, requester submission, finance change requests, resubmission, stage advancement, final approval/rejection, history and persistent notice read state
Panel browser journey	Attachment byte fidelity/removal, comments after reload, related navigation and activity
Accessibility and native	Targeted workflow/dialog audits found no serious/critical issues in those journeys; Linux native lifecycle was checked separately
Public smoke checks	Both hosts passed release identity, authenticated approval inbox/configuration availability and record controls; no live approval writes or runtime errors in that journey

These are dated development results. They are not an exhaustive certification of every page, device, theme, backend or tenant. The PDF task verifies its own document artifact; it does not turn historic test results into a new full application test run.

Troubleshooting

Symptom	Likely explanation	What to do
Signed out after a release	Demo API sessions were reset by its restart	Sign in again; acknowledged service drafts remain available
Save says conflict	Another write advanced the record/draft version	Review current saved values and explicitly choose the next action
A lookup is disabled	Its reference-data request failed	Read the named failure and retry; do not assume there are no configured options
CSV field cannot validate	Missing mapping, unsupported value, format or duplicate	Review row errors, fix mapping/source values and validate again

Symptom	Likely explanation	What to do
CSV stopped with pending rows	It was closed/paused or a request was interrupted	Reopen the latest job and Resume import
Approval action is disabled	Wrong role/stage, self-request, no comment/selection or changed record	Check stage, ownership, comment and saved-version warning
Approval result changed elsewhere	This view is stale	Refresh approvals and review again
Earlier approved record now warns	Saved values differ from the submitted version	Original requester saves corrections and resubmits
No notice in the header bell	Approval notices are separate	Open My approval notifications; global synchronization is pending
Local check calls a public API	A build-time API URL was compiled into the bundle	Rebuild/start with the isolated API and inspect actual browser requests
Utility-class verificatio n fails after a UI edit	Local CSS artifacts may predate new classes	Build the matching source, then rerun artifact verification

19. Source map, glossary and document maintenance

Where to read or extend the implementation

Paths below are relative to desktop-clients unless marked repository root. They identify the inspected working tree; some developed source was not yet committed when this guide was requested.

Path	Read it for
products/active.ts; products/provider.tsx	Active product and composition in both shells
products/ledger; scripts/products/create-product.mjs	Second profile and generator
packages/erp-config/src/product.ts	Branding/page/action configuration rules
packages/erp-config/src/entity-schemas. ts; form-rules.ts; imports.ts	Registered schemas, validation and import definition
packages/erp-screens/src/product-servic es.tsx	Injectable product service boundary
packages/erp-data/src/records.ts	Editor controller and versioned record/draft protocol

Path	Read it for
packages/erp-screens/src/worklist/worklist-page.tsx	Worklist behavior and workflow entry points
packages/erp-screens/src/records/record-panels.tsx	Supporting record panels
packages/erp-data/src/csv-import.ts; packages/erp-screens/src/imports	CSV contract/parser and workflow UI
packages/erp-data/src/approvals.ts; packages/erp-screens/src/approvals	Approval contract and workspace
packages/erp-screens/src/page-renderer.tsx	Shared page composition and record wrappers
packages/workspace-core; packages/platform-ports	Desktop state ownership and platform abstraction
scripts/deploy.mjs; scripts/deployment	Isolated artifact preparation/activation
dummy-api/server.mjs and *-store.mjs (repository root)	Demo routing, permission/version checks and persisted state
e2e/records.mjs, workflows.mjs, record-panels.mjs, imports.mjs, approvals.mjs	Reproducible isolated feature journeys

Supporting notes are docs/frontend-product-review.md, desktop-development-plan.md, desktop-reliability.md, record-editing.md, worklist-workflows.md, product-profiles.md, record-panels.md, csv-import.md, approvals.md, deployment.md, running-tauri.md, remaining-work.md, ai-hardening-ledger.md and ai-service-contract.md. Their dated totals and release statements may describe an earlier milestone. This guide consolidates them as of 8 September 2026.

Plain-language glossary

Term	Meaning
Adapter	Code translating a shared frontend contract to an application's service
Shell	The surrounding application frame: navigation, header, windows and routing
Schema	A definition of fields, types, choices and validation rules
Tenant	One customer organization using the application
Recovery draft	A service-acknowledged copy of unfinished edits, separate from a saved record

Term	Meaning
Version conflict	A refusal because the user acted on an older saved revision
Idempotency	Repeating the same identified request does not repeat its effect
Partial result	Some selected records succeeded while others failed
Snapshot	A captured state or version used for saving, review or a release
Release activation	Selecting a built artifact and restarting the frontend services on it
Smoke check	A small check that a deployed workflow loads and answers correctly

Updating this guide

Edit frontend-platform-guide.md in this folder, then regenerate its PDF using build_pdf.py and the pinned documentation requirements. Keep the Markdown as the editable source and the PDF as the shareable output. Update the reporting date, release, completion map, test scope and pending items together. Review every rendered page for clipping and broken tables before committing.

A documentation commit is separate from committing application source and from activating a release. The final task handover should identify exactly which files were committed/pushed. Do not infer source publication from the fact that a deployed demonstration works.